Machine Learning Assignment 1

EECE5644 - Fall 2026 | David Abu | Reviewed working draft
Regenerated October 16, 2026. Mathematical analysis, experimental results, and code are included in this PDF.

Results at a glance

Experiment	Error estimate	Meaning
Q1: true Gaussian / Naive Bayes / LDA	2.593% / 3.248% / 5.210%	Same 10,000 generated observations
Q2: 0-1 / class-3 penalty 10 / 100	17.49% / 27.57% / 43.87%	Higher penalties protect class 3
Q3: white wine / HAR	50.327% / 0.301%	Required all-sample training errors

External resources and AI assistance

The supplied draft and initial Python implementation were generated with Claude (Anthropic), as identified in the supplied materials. OpenAI Codex reviewed that implementation, corrected ROC integration and validation preprocessing, downloaded original UCI data, reran the experiments, performed numerical checks, and generated this revised report and study guide. The exact Claude model stated in the original draft has not been independently verified. This is an AI-assisted working draft for David to review and understand; it does not assert that David has already verified or independently authored every derivation or line of code.

Software: Python, NumPy, SciPy, pandas, Matplotlib, and ReportLab. Assignment requirements and all numerical parameter values for Q1 come from the instructor's Assignment 1. Dataset sources, hashes, and reproducibility instructions appear later in this report and in the accompanying package.

Notation

L denotes the true label, D the classifier decision, and lambda(i,j) the loss incurred by deciding i when the truth is j. Confusion matrices use decision rows and true-label columns; each column sums to one. A density p(x|L=j) is distinct from a posterior class probability P(L=j|x).

Scope of verification

All three experiments were rerun locally. 22 numerical checks passed, including agreement with SciPy Gaussian log-densities, an independent rank-based AUC calculation, confusion-matrix totals, and the closed-form LDA direction. Training performance is distinguished from validation performance throughout.

Question 1 | Model and minimum-risk rule

Generate N=10,000 observations once with NumPy seed 5644. Draw L=0 with probability 0.35 and L=1 with probability 0.65, then draw X from its class Gaussian. The realized counts are N0=3460 and N1=6540. Save X and L in out/q1_data.npz and reuse them in Parts A, B, and C.

```
m0 = (-0.5, -0.5, -0.5, -0.5)^T     m1 = (1, 1, 1, 1)^T
C0 = 0.25 * [[ 2,  -0.5,  0.3, 0],
             [-0.5, 1,   -0.5, 0],
             [ 0.3,-0.5,  1,   0],
             [ 0,   0,    0,   2]]
C1 = [[ 1,   0.3, -0.2, 0],
      [ 0.3, 2,    0.3, 0],
      [-0.2, 0.3,  1,   0],
      [ 0,   0,    0,   3]]
```

Part A.1: derive the decision rule

$$R(i \mid x) = \sum_{j=0}^{1} \lambda_{ij} P(L = j \mid x), \qquad D(x) = \arg\min_i R(i \mid x)$$

Decide 1 when R(1|x) < R(0|x). Substitute Bayes' rule and cancel the positive marginal density p(x):

$$(\lambda_{01} - \lambda_{11}) p(x \mid 1) P_1 > (\lambda_{10} - \lambda_{00}) p(x \mid 0) P_0$$

When lambda01 > lambda11, divide to obtain the requested likelihood-ratio test. For ordinary error costs, lambda10 >= lambda00 as well, so the threshold is nonnegative.

$$\frac{p(x \mid 1)}{p(x \mid 0)} > \gamma, \qquad \gamma = \frac{(\lambda_{10} - \lambda_{00}) P_0}{(\lambda_{01} - \lambda_{11}) P_1}$$

For arbitrary nonnegative losses, the undivided inequality remains valid: if lambda01-lambda11 is negative, reverse the inequality after division; if it is zero, compare the remaining term directly. Ties have equal conditional risk and may be broken consistently.

Under 0-1 loss, both correct decisions cost 0 and both mistakes cost 1. Therefore gamma*=0.35/0.65=0.5384615, and log(gamma*)=-0.6190392. This is minimum-error classification, also called the MAP class decision.

Numerically stable Gaussian scoring

$$\log g(x \mid m, C) = -\tfrac{1}{2}\left[d \log(2\pi) + \log|C| + (x - m)^T C^{-1} (x - m) \right]$$

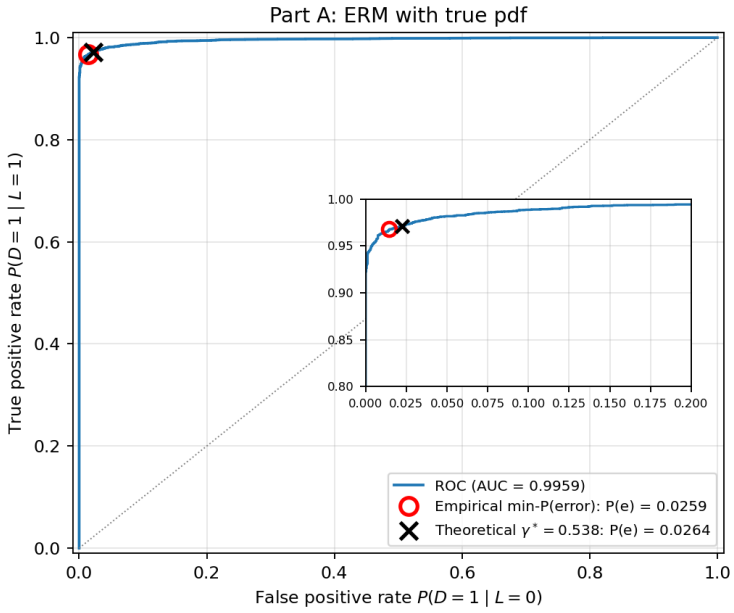
Compute s(x)=log g(x|m1,C1)-log g(x|m0,C0), then decide 1 if s(x)>log(gamma). Cholesky factorization supplies the log-determinant and Mahalanobis distance without explicitly inverting C. Unequal covariances make the decision boundary quadratic.

Question 1 | ROC and minimum-error operating points

For each threshold, estimate TPR as the fraction of true class-1 samples decided 1 and FPR as the fraction of true class-0 samples decided 1. Sort scores, process all equal scores together, and use the strict rule score > threshold. The initial threshold -infinity gives (FPR,TPR)=(1,1); the maximum observed score gives (0,0).

$$\widehat{P}_e(\gamma) = 0.35\,\widehat{\mathrm{FPR}(\gamma)} + 0.65\,[1 - \widehat{\mathrm{TPR}(\gamma)}]$$

Classifier	Threshold	TPR	FPR	Error	AUC
True pdf	gamma=0.7907	0.9679	0.0145	0.02593	0.99589
Naive Bayes	gamma=0.6396	0.9654	0.0286	0.03248	0.99420
Fisher LDA	tau=-0.00533	0.9512	0.0584	0.05210	0.98536



True-distribution ROC. Red circle: empirical minimum; black cross: prior-derived threshold.

At gamma*=0.53846 the estimated error is 0.026376; the empirical minimum is 0.025929 at gamma=0.790702. The empirical minimizer lies in [0.782880, 0.798602); every threshold in this interval yields the same decisions.

An independent 2,000,000-observation Monte Carlo run gives Bayes-rule error 0.0262235. Its binomial standard error is 0.0001130 (approximate 95% Monte Carlo interval: estimate +/- 0.0002215). This is an estimate, not an exact analytic Bayes error.

The threshold selected on 10K observations adapts to sampling noise and its reported minimum is optimistically selected on those observations. It does not disprove the population optimum gamma*. In Parts A and B, errors use the specified priors rather than the slightly different realized class proportions.

Question 1 | Naive Bayes and Fisher LDA

Part B: incorrect independence model

Keep the data unchanged, but score it with diagonal covariances: C0_NB=diag(0.5,0.25,0.25,0.5) and C1_NB=diag(1,2,1,3). These Gaussian models retain true means and variances but discard correlations. Repeat the same ROC sweep and prior-weighted error calculation.

The best empirical error rises from 0.02593 to 0.03248, a 25.2% relative increase. AUC falls from 0.99589 to 0.99420. At gamma*=0.53846 the NB error is 0.03260; that threshold is optimal for the assumed model, not guaranteed optimal for the actual distribution.

Part C: estimate moments and learn one projection

$$\hat{m}_j = \frac{1}{N_j} \sum_{n\,:\,L_n = j} x_n, \qquad \hat{C}_j = \frac{1}{N_j} \sum_{n\,:\,L_n = j} (x_n - \hat{m}_j)(x_n - \hat{m}_j)^T$$

Use equal class weights in the scatter matrices as instructed. Choose SB=(m1_hat-m0_hat)(m1_hat-m0_hat)^T and SW=C0_hat+C1_hat. Alternative common scalar normalizations change eigenvalues, but not the maximizing direction.

$$J(w) = \frac{w^T S_B w}{w^T S_W w}, \qquad S_B w = \lambda S_W w, \qquad w \propto S_W^{-1}(\hat{m}_1 - \hat{m}_0)$$

Choose the generalized eigenvector for the largest eigenvalue, normalize it to unit length, and orient its sign so that class 1 has the larger projected mean.

```
m0_hat: [-0.49969, -0.50477, -0.49605, -0.49799]
m1_hat: [1.01096, 1.01602, 0.99471, 0.98294]
w_LDA:  [0.60278, 0.27468, 0.70857, 0.24318]
```

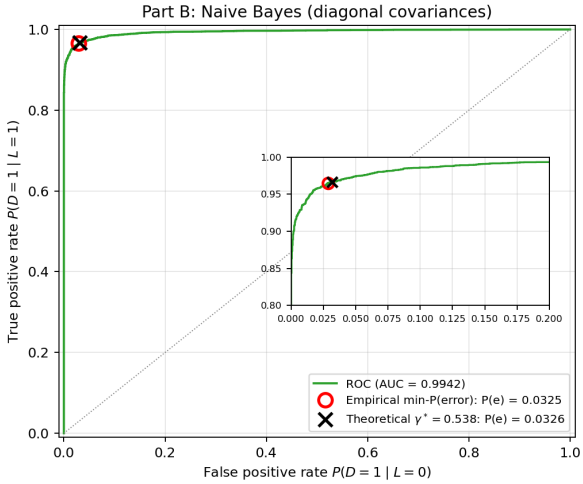
Estimated within-class covariance matrices

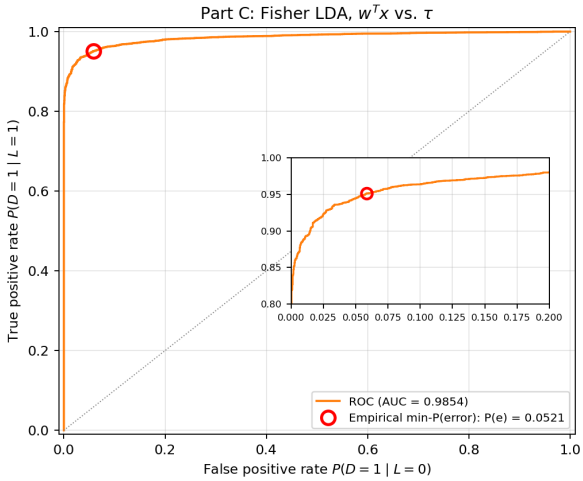
```
C0_hat =
0.5123  -0.1269  0.0773  0.0099
-0.1269  0.2461  -0.1249  -0.0051
0.0773  -0.1249  0.2580  0.0110
0.0099  -0.0051  0.0110  0.5068

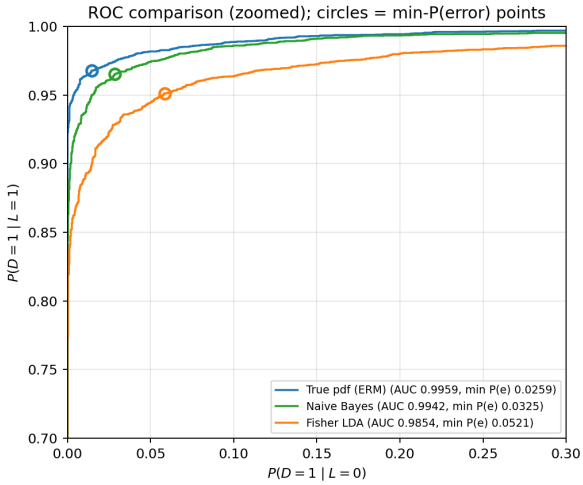
C1_hat =
1.0079  0.3277  -0.2083  0.0136
0.3277  2.0730  0.3098  0.0099
-0.2083  0.3098  1.0147  -0.0148
0.0136  0.0099  -0.0148  3.0055
```

The largest generalized eigenvalue is 4.688517 under this normalization. The eigenvector agrees with the normalized closed-form direction. Classify using w^T x > tau, sweeping all projected scores; select tau=-0.0053334. For LDA use the sample-count error as requested: errors/N=0.0521.

Question 1 | Comparison and interpretation

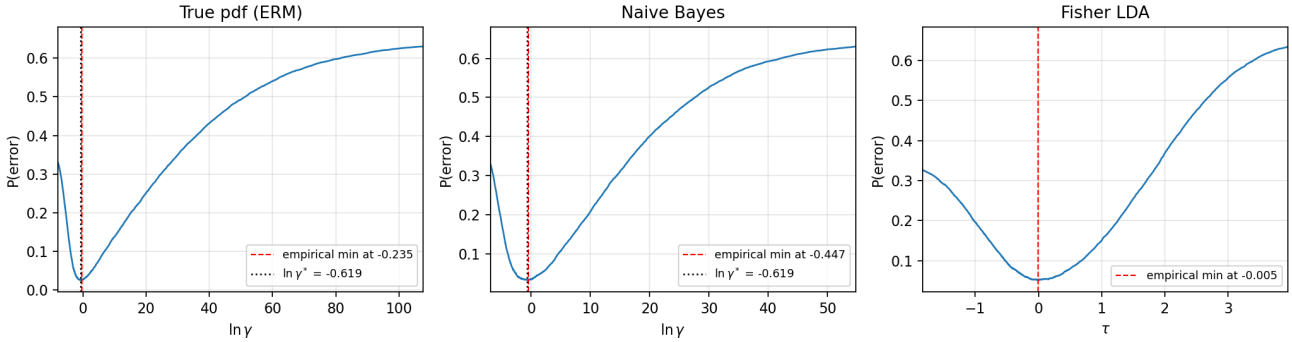






Top: Naive Bayes ROC. Middle: full Fisher LDA ROC. Bottom: comparison with empirical minimum-error operating points marked.

Question 1 | Why the thresholds and classifiers differ



The true-pdf rule performs best because it uses the correct joint density and priors. Naive Bayes loses information about correlations but retains different class variances, so its boundary remains quadratic. Fisher LDA uses one linear projection and a single threshold, yielding a hyperplane; it cannot reproduce the curved Bayes boundary in this unequal-covariance example.

Fisher's criterion optimizes a separation-to-spread ratio. It is not generally an error-minimization objective for arbitrary distributions. Equal-covariance Gaussian classes give a Bayes discriminant in the Fisher direction with the appropriate threshold. Equal weights are used here to match the question; priors still affect operating-point selection. Estimated ROCs can cross locally through finite-sample variation, so the observed ordering should not be claimed to hold at every possible threshold.

Question 2 | Construct a three-class mixture

Choose four 3D Gaussian components with means at the vertices of a regular tetrahedron of edge 2.5. Their average standard deviation, defined as the average of sqrt(trace(Ck)/3), is 0.991453. Thus every pairwise mean distance is about 2.522 times this scale, creating overlap.

```
Component means:
0.0000  0.0000  0.0000
2.5000  0.0000  0.0000
1.2500  2.1651  0.0000
1.2500  0.7217  2.0412

C1 =
1.00  0.30  -0.20
0.30  0.80  0.10
-0.20  0.10  1.20

C2 =
0.90  -0.20  0.10
-0.20  1.10  0.30
0.10  0.30  0.80

C3 =
1.20  0.20  0.00
0.20  0.90  -0.30
0.00  -0.30  1.00

C4 =
0.80  0.00  0.20
0.00  1.00  0.00
0.20  0.00  1.10
```

The class priors are (0.3,0.3,0.4). Class 1 uses component 1, class 2 uses component 2, and class 3 is an equal mixture of components 3 and 4:

$$p(x \mid 3) = \frac{1}{2} g(x \mid \mu_3, \Sigma_3) + \frac{1}{2} g(x \mid \mu_4, \Sigma_4)$$

For N=10,000, seed 2026 gives class counts [2950, 2993, 4057]. Draw a label first; for class 3 select either component with probability 0.5; draw its Gaussian observation. Save data and component identifiers in out/q2_data.npz.

Part A: minimum probability of error

$$D(x) = \arg\max_j P(L=j \mid x) = \arg\max_j \left[\log p(x \mid j) + \log P_j \right]$$

For class 3, evaluate log(0.5*g3+0.5*g4) using log-sum-exp. The mixture density is not a Gaussian centered at the average component mean. Under 0-1 loss, conditional risk is 1-P(L=i|x), so maximizing the posterior minimizes risk.

Question 2 | MAP results and data visualization

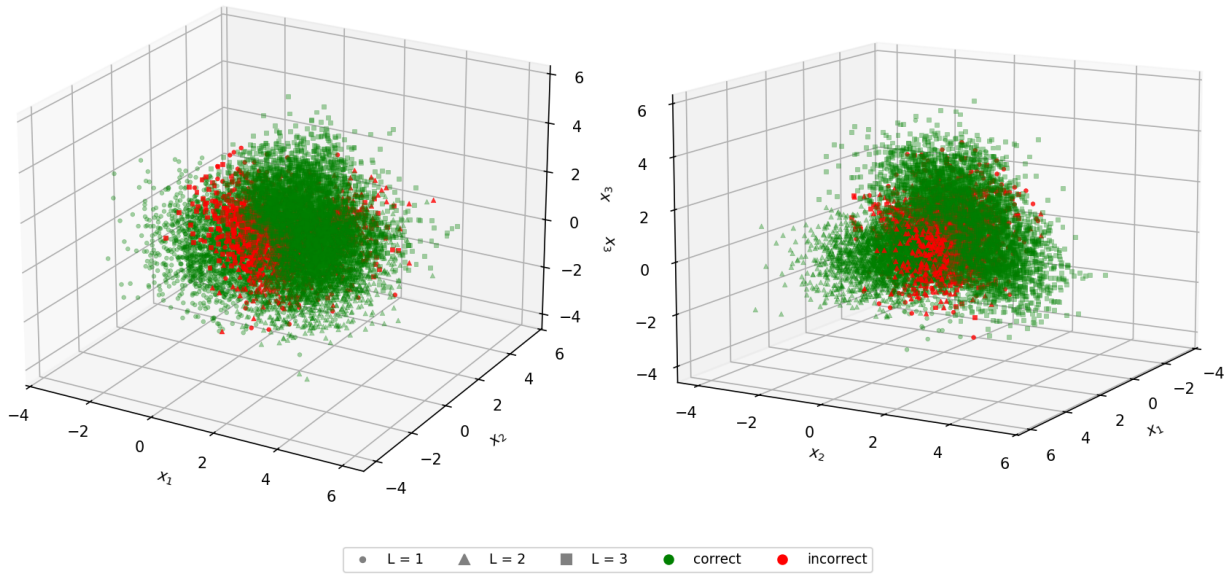
Decision / true label	1	2	3
1	0.7963	0.0672	0.0845
2	0.0600	0.8199	0.0656
3	0.1437	0.1129	0.8499

Each entry estimates P(D=i|L=j) by dividing the corresponding decision/true-label count by the number of observations in true class j. Columns sum to one.

```
Decision/true-label count matrix:
2349  201  343
177  2454  266
424  338  3448
```

MAP error = 0.1749; this equals the observed mean 0-1 loss. The average conditional minimum risk is 0.1775; its difference from realized loss reflects finite sampling.

MAP (0-1 loss): P(error) = 0.1749, estimated min. expected risk = 0.1749



Two views of the same 10K observations. True class is encoded by marker shape (1: dot, 2: triangle, 3: square); green is correct and red is incorrect.

Overlapping Gaussian components make some errors unavoidable even with the true model. Component identity and class identity differ: both components 3 and 4 count as class 3, so confusing those components is not a classification error.

Question 2 | Unequal costs and expected risk

```
Lambda_10  = [[0,1,10], [1,0,10], [1,1,0]]
Lambda_100 = [[0,1,100],[1,0,100],[1,1,0]]
```

Columns represent true labels. A class-3 observation assigned label 1 or 2 costs w=10 or w=100; mistakes on true classes 1 or 2 cost 1. Define pj=P(L=j|x).

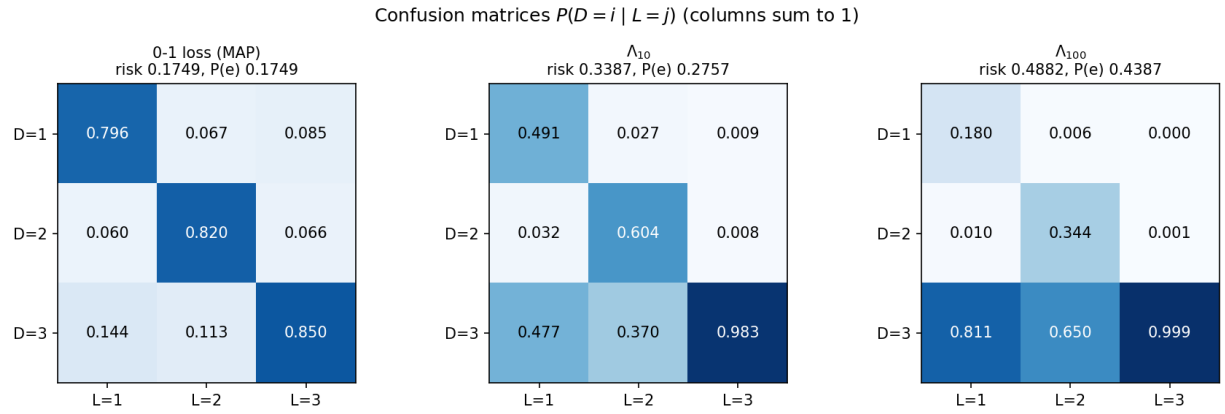
$$R(1 \mid x) = p_2 + wp_3, \quad R(2 \mid x) = p_1 + wp_3, \quad R(3 \mid x) = p_1 + p_2$$

$$D(x) = \arg\min_i \sum_j \lambda_{ij} P(L = j \mid x)$$

Class 3 is preferred when 1-p3 < min(p1,p2)+w*p3, equivalently p3 > [1-min(p1,p2)]/(w+1), apart from ties. Increasing w expands the region assigned class 3. This is a posterior-dependent condition, not a fixed standalone threshold independent of p1 and p2.

Loss	Observed risk	Conditional-risk average	Error	Class-3 recall
0-1	0.1749	0.1775	0.1749	0.8499
Lambda10	0.3387	0.3451	0.2757	0.9827
Lambda100	0.4882	0.4899	0.4387	0.9988

Observed risk is the sample mean of lambda(Dn,Ln). The conditional-risk average is the sample mean of min_i R(i|xn), using the known generating distribution. These are two Monte Carlo estimators of the same population risk; they need not match exactly on one dataset.

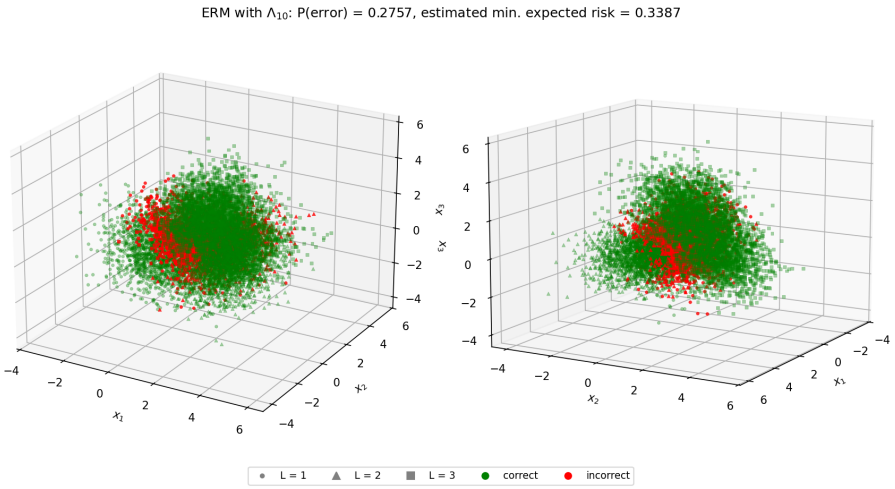


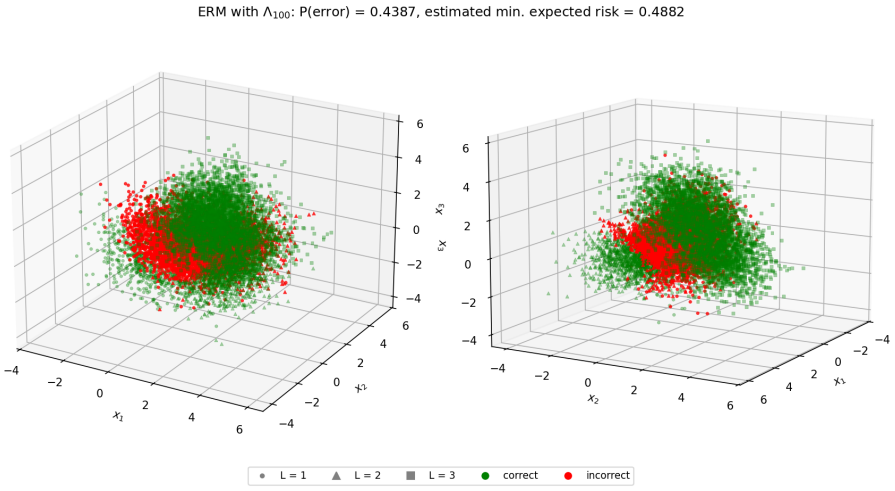
Decision rows and true-label columns. The class-3 diagonal increases as its missed-detection penalty grows.

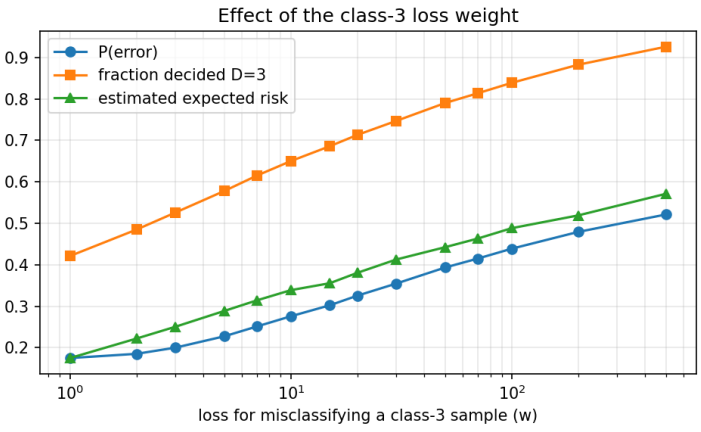
Tradeoff

As the penalty increases, class-3 recall rises from 0.8499 to 0.9827 to 0.9988, while overall error rises from 0.1749 to 0.2757 to 0.4387. More observations from classes 1 and 2 are deliberately assigned class 3 to avoid expensive class-3 misses. Minimum expected risk is not the same objective as minimum error when losses differ.

Question 2 | How changing the penalty changes decisions







Top: penalty 10. Middle: penalty 100. Bottom: behavior across increasing penalties. All experiments reuse the same data.

For penalty 100, 5 true class-3 observations are missed, contributing 0.0500 to observed risk. The 4335 class-1/2 observations assigned class 3 contribute 0.4335; the 47 class-1/2 cross-errors contribute 0.0047. Total: 0.4882. As w tends to infinity, the limiting rule assigns class 3 almost everywhere, with population risk P(L=1)+P(L=2)=0.6. The observed finite-sample limit is the realized fraction of classes 1 and 2.

Question 3 | Estimation, preprocessing, and regularization

Fit one full-covariance Gaussian per observed class. Samples are treated as identically distributed independent observations for estimation, although HAR windows and repeated subjects may violate independence in practice. Class means, covariances, and priors use all available samples for the required final training evaluation.

$$\hat{P}_j = N_j/N, \quad \hat{m}_j = N_j^{-1} \sum_{n:L_n=j} x_n, \quad \hat{C}_j = N_j^{-1} \sum_{n:L_n=j} (x_n - \hat{m}_j)(x_n - \hat{m}_j)^T$$

Standardize features using their mean and standard deviation. For all-sample evaluation, fit this transformation on all samples. For wine cross-validation, fit it separately on each training fold; for HAR validation, fit it only on the official training portion. Apply the fitted transformation unchanged to the corresponding validation portion.

Without regularization, invertible affine preprocessing preserves Gaussian discriminant decisions because the same Jacobian factor cancels between classes. Adding lambda*I is not affine invariant; standardization gives comparable units before adding isotropic variance.

$$C_{j,\mathrm{reg}} = \hat{C}_j + \lambda_j I, \qquad \lambda_j = \alpha \frac{\mathrm{tr}(\hat{C}_j)}{\mathrm{rank}(\hat{C}_j)}$$

A positive lambda shifts every covariance eigenvalue upward; for a singular positive semidefinite estimate, the smallest eigenvalue becomes lambda (not strictly greater). Numerical rank uses an eigenvalue tolerance proportional to matrix size and machine precision. This prevents singular Gaussian calculations and controls unstable low-variance directions.

$$D(x) = \arg\max_j \left[-\tfrac{1}{2}(x-\hat{m}_j)^T C_{j,\mathrm{reg}}^{-1}(x-\hat{m}_j) - \tfrac{1}{2}\log|C_{j,\mathrm{reg}}| + \log \hat{P}_j \right]$$

Regularization selection

Wine uses stratified five-fold CV and selects alpha=0.001. HAR uses the official 7352/2947 split as validation and selects alpha=0.0001. Because that split selects alpha, its reported error is a tuning/validation result, not an untouched final-test estimate. After selection, refit on all samples as required by the assignment.

alpha	Wine training	Wine CV	HAR training	HAR validation
1e-06	0.5002	0.5155	0.0217	0.0845
1e-05	0.5002	0.5155	0.0118	0.0821
0.0001	0.5008	0.5157	0.0030	0.0818
0.001	0.5033	0.5149	0.0149	0.1035
0.01	0.5043	0.5184	0.0652	0.1256
0.03	0.5065	0.5214	0.0938	0.1354
0.1	0.5159	0.5296	0.1482	0.1927
0.3	0.5208	0.5300	0.2633	0.3156

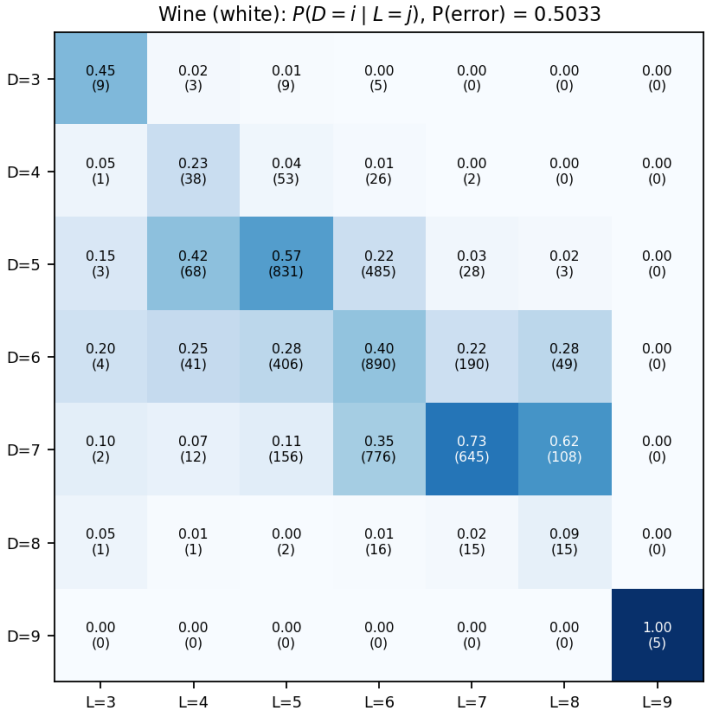
Neither training nor validation error is guaranteed to vary monotonically with regularization. The observed HAR training sweep is nonmonotonic, so smaller alpha cannot simply be described as always reducing training error.

Question 3 | White-wine results

The UCI white-wine dataset has 4898 observations, 11 features, and seven observed quality labels: 3 through 9. Scores 0-2 and 10 have no observations and are not fitted classes. Use the white dataset rather than combining red and white wine, matching the stated sample count.

Class	N	Prior	Rank	lambda	cond(reg)
3	20	0.0041	11	3.47e-03	1.07e+03
4	163	0.0333	11	1.36e-03	1.47e+02
5	1457	0.2975	11	9.93e-04	2.03e+02
6	2198	0.4488	11	9.37e-04	1.07e+02
7	880	0.1797	11	7.71e-04	2.83e+02
8	175	0.0357	11	8.86e-04	4.74e+02
9	5	0.0010	4	1.28e-03	2.54e+03

At alpha=0.001, the required all-sample error is 0.5032666 (50.327%). Always predicting quality 6 gives error 0.5512; the fitted model improves exact-grade classification modestly. Within one quality grade, accuracy is 0.9265.



Every cell reports P(D=i|L=j) and its raw count. Most confusion occurs among neighboring quality labels.

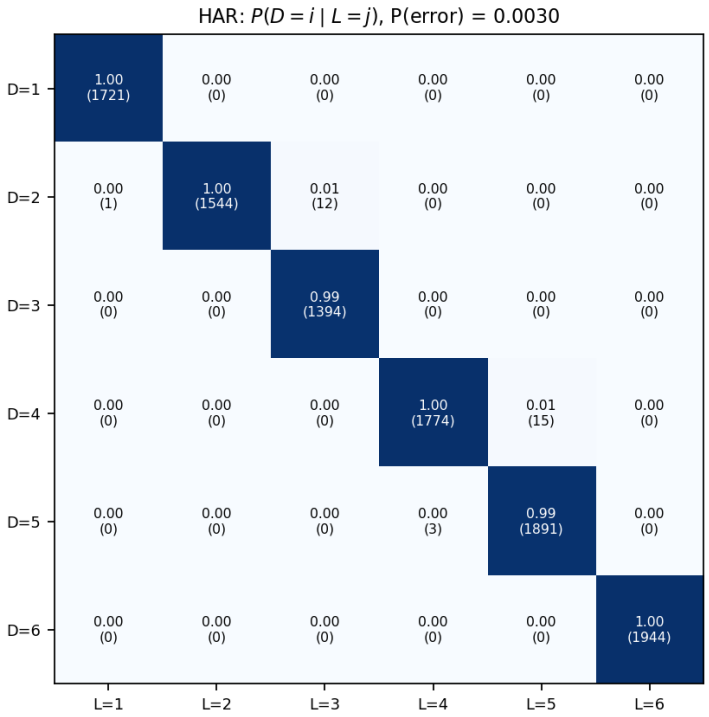
Quality 9 has only five observations in 11 dimensions; its sample covariance rank is at most four. Its perfect training classification is therefore weak evidence of performance on new quality-9 wines. Class imbalance and overlapping chemical measurements make exact quality prediction difficult under a single Gaussian per class.

Question 3 | Human activity recognition results

Combine the original 7352 training and 2947 test observations for the required final fit and training evaluation: N=10299, d=561, six activity labels. The official split is retained only for the separate alpha-validation experiment.

Class	N	Prior	Rank	lambda	cond(reg)
1	1722	0.1672	468	7.35e-05	1.15e+06
2	1544	0.1499	468	6.48e-05	9.19e+05
3	1406	0.1365	469	1.21e-04	1.50e+06
4	1777	0.1725	468	3.80e-05	1.15e+06
5	1906	0.1851	468	3.22e-05	9.57e+05
6	1944	0.1888	468	4.96e-05	1.05e+06

At alpha=0.0001, all-sample error is 0.0030100, or 31/10299 observations. The selected official-split validation error is 0.0817781 (8.18%). These answer different questions and must not be equated.

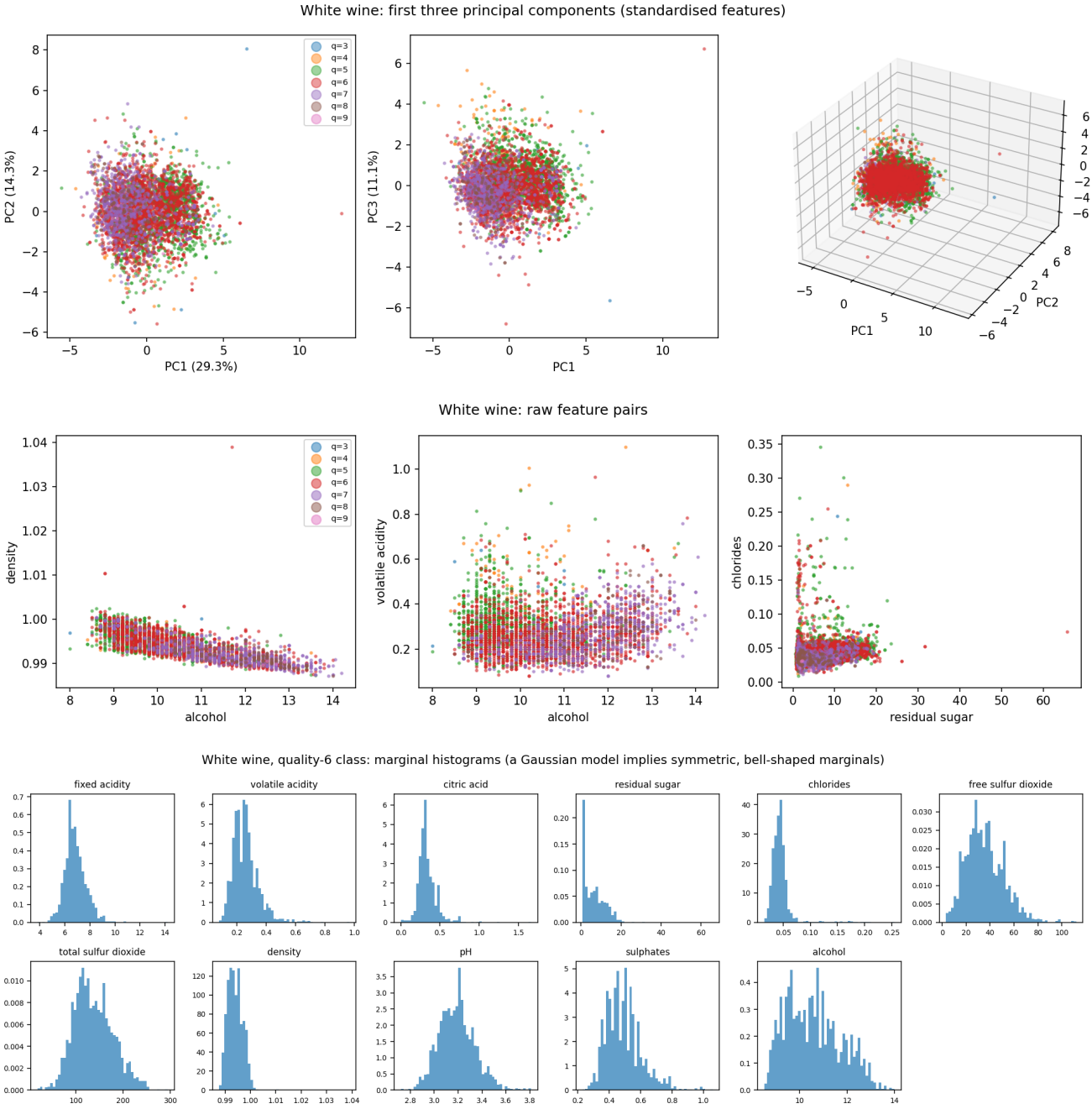


All-sample confusion matrix. Errors concentrate in sitting versus standing and walking downstairs versus upstairs.

Estimated covariance ranks are approximately 468-469 rather than 561, with very large raw condition numbers. This establishes numerical dependence or near-dependence among features. It does not, by itself, identify which engineered features cause it: nonlinear deterministic relationships alone do not imply linear covariance rank deficiency. Regularization permits stable density evaluation in all 561 coordinates.

High-dimensional class-specific covariance models are flexible, and the same observations are used for fitting and scoring. The 0.301% training error therefore cannot establish near-perfect accuracy on new subjects. The much larger validation error illustrates this limitation.

Question 3 | Wine projections and Gaussian assumptions

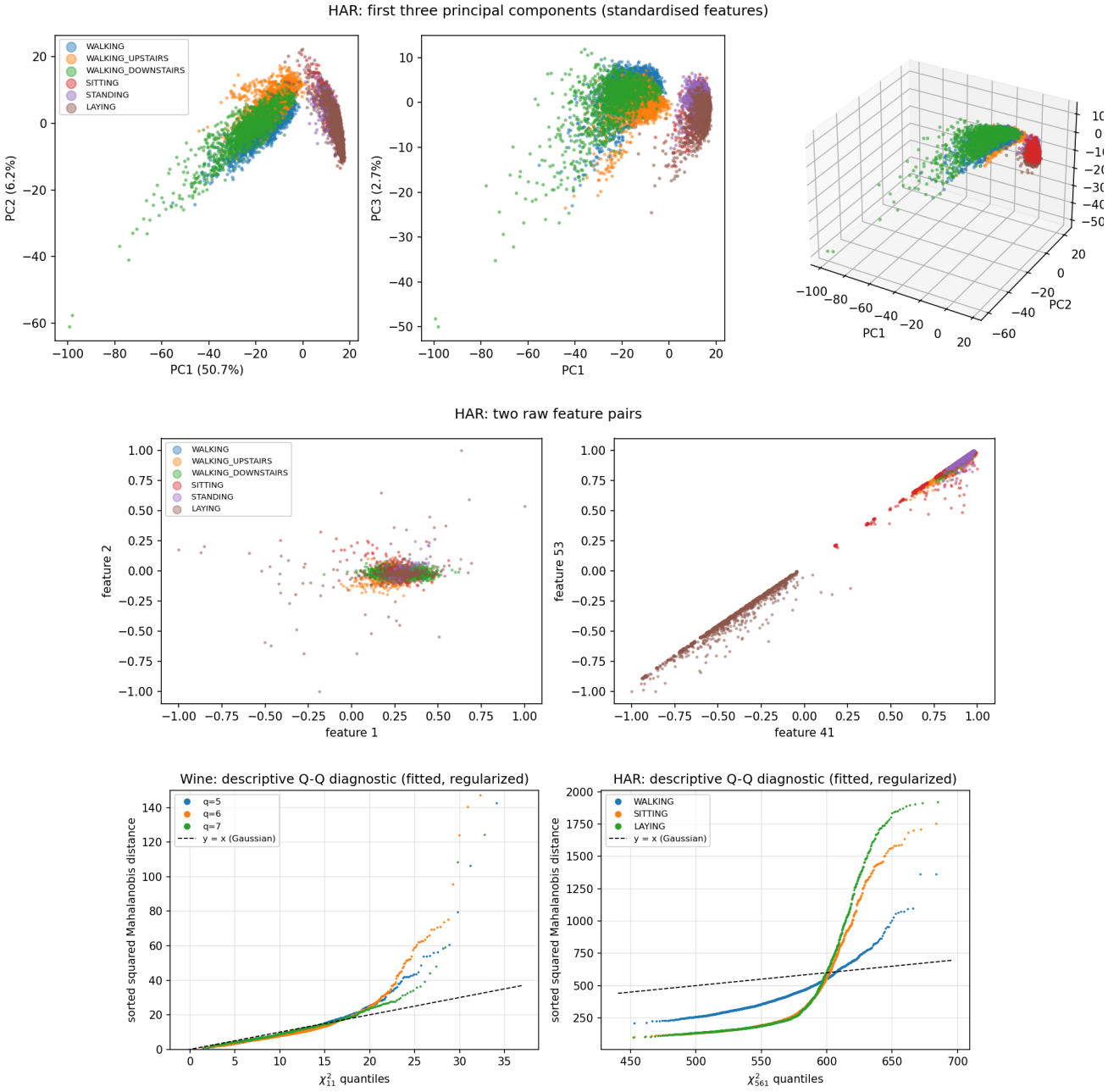


Top: first three PCA coordinates. Middle: selected raw feature pairs. Bottom: quality-6 marginal distributions.

PCA is computed on centered standardized features for visualization; the classifier still uses all 11 features. The first three components explain 29.29%, 14.32%, and 11.11% of total standardized variance.

Strong overlap between quality classes is visible in the projections. Several within-class marginals have skewness and long tails rather than symmetric bell shapes. A multivariate Gaussian requires Gaussian marginals, so these patterns challenge that assumption. Conversely, approximately Gaussian marginals alone would not establish a Gaussian joint distribution. Model mismatch can contribute to neighboring-quality confusion; these plots do not isolate its contribution from intrinsic overlap and label uncertainty.

Question 3 | HAR projections and diagnostic limits



PCA and raw-feature views, followed by descriptive squared-Mahalanobis Q-Q plots for selected classes.

For HAR, PC1 explains 50.74% and PCs 1-3 together explain 59.67% of variance. Low-dimensional plots show structure and overlap but cannot determine whether all 561-dimensional classes are separable.

With known, full-rank Gaussian parameters in d dimensions, squared Mahalanobis distance follows chi-square(d). Here parameters are fitted on the same samples and covariances are regularized, with substantial rank deficiency in HAR. The nominal chi-square reference is consequently not a calibrated goodness-of-fit test. Deviations are descriptive and can reflect estimation, shrinkage, effective dimension, and non-Gaussianity. They should not be presented as standalone proof of non-Gaussian classes.

Reproducibility, source records, and review notes

How to rerun

Use Python 3.10+ with NumPy 2+, SciPy, pandas, Matplotlib, and Pillow. From the extracted project folder run: python run_all.py. The runner downloads original UCI archives if needed, creates output folders, runs q1.py, q2.py and q3.py, and checks results. Raw datasets are not bundled in the compact code package; the download script recreates them. PDF generation additionally needs ReportLab and suitable fonts.

Source records

Instructor: EECE5644 Fall 2026, Assignment 1, supplied PDF. Dataset providers:

UCI Wine Quality, dataset 186
UCI Human Activity Recognition Using Smartphones, dataset 240

wine: 91353 archive bytes. SHA-256:

`3ed56667f4b828242bd732d7d1dd7f2861e54432239d7fa63877014cbb0304d4`

har: 61005872 archive bytes. SHA-256:

`c00b803081a5c797cd5e4b83700a9810b38d53d9d84e01917e090e1fdbc81031`

Archives were downloaded from the original UCI static endpoints on October 16, 2026. The earlier draft's mirror identity claim is not reused; matching dimensions alone does not establish byte identity. download_data.py and data/sources.json record the exact sources used for this rerun.

Changes relative to the supplied draft

- Replaced the machine-specific /home/claude/data path with a project-relative path and provided a one-command runner and data downloader.

- Corrected AUC integration to preserve the order of vertical ROC segments; checked it against an independent rank statistic. The three AUCs round to 0.99589, 0.99420, and 0.98536.

- Fitted validation preprocessing only on the training portion. Kept the required all-sample estimates separate from validation estimates.

- Qualified the HAR alpha-selection split as validation; removed unsupported claims about exact model provenance, covariance dependence causes, monotonic training-error behavior, and calibrated Q-Q testing.

- Saved ROC thresholds/rates, data, numerical summaries, and verification records. Added the following executable source appendix.

The unchanged uploaded PDF and ZIP remain the originals. This report is a revised working draft; David should review the reasoning, results, and AI acknowledgment before submitting. No submission or upload to Canvas has been performed.

Source appendix | common.py

```python
"""Shared helpers for EECE5644 Assignment 1."""
import numpy as np
from scipy.linalg import solve_triangular


def gauss_logpdf(X, m, C):
    """Log of the multivariate Gaussian pdf g(x|m,C), evaluated row-wise on X (N x d).
    Uses a Cholesky factorisation for numerical stability."""
    X = np.atleast_2d(X)
    d = X.shape[1]
    L = np.linalg.cholesky(C)
    Z = solve_triangular(L, (X - m).T, lower=True, check_finite=False)          # d x N, Z = L^{-1}(x - m)
    maha = np.sum(Z ** 2, axis=0)              # (x-m)^T C^{-1} (x-m)
    logdet = 2.0 * np.sum(np.log(np.diag(L)))
    return -0.5 * (d * np.log(2 * np.pi) + logdet + maha)


def roc_sweep(score, labels):
    """Sweep a threshold t over all observed score values for the rule D = 1 iff score > t.
    Returns thresholds (starting at -inf, so the curve starts at (1,1)), P(D=1|L=0) and P(D=1|L=1).
    The largest observed score gives the finite end point (0,0)."""
    score = np.asarray(score, float)
    labels = np.asarray(labels).astype(int)
    n1, n0 = labels.sum(), (1 - labels).sum()
    order = np.argsort(score)
    s_sorted = score[order]
    l_sorted = labels[order]
    # number of samples with score <= s_sorted[k]  -> those are decided 0 at threshold s_sorted[k]
    c1_le = np.cumsum(l_sorted)
    c0_le = np.cumsum(1 - l_sorted)
    # keep only the last index of each run of tied scores
    last = np.r_[s_sorted[1:] != s_sorted[:-1], True]
    thr = np.r_[-np.inf, s_sorted[last]]
    tpr = np.r_[1.0, (n1 - c1_le[last]) / n1]
    fpr = np.r_[1.0, (n0 - c0_le[last]) / n0]
    return thr, fpr, tpr


def auc(fpr, tpr):
    # Preserve the order of vertical ROC segments (tied false-positive rates).
    return -np.trapezoid(tpr, fpr)
```

Source appendix | q1.py

```python
"""Question 1: ERM / LRT with true pdf, Naive Bayes, and Fisher LDA on a 2-class 4-D Gaussian problem."""
import json
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy.linalg import eigh
from common import gauss_logpdf, roc_sweep, auc

rng = np.random.default_rng(5644)
N = 10000
P = np.array([0.35, 0.65])
m0 = -0.5 * np.ones(4)
C0 = 0.25 * np.array([[2, -0.5, 0.3, 0],
                      [-0.5, 1, -0.5, 0],
                      [0.3, -0.5, 1, 0],
                      [0, 0, 0, 2]])
m1 = np.ones(4)
C1 = np.array([[1, 0.3, -0.2, 0],
               [0.3, 2, 0.3, 0],
               [-0.2, 0.3, 1, 0],
               [0, 0, 0, 3]])
for C in (C0, C1):
    assert np.all(np.linalg.eigvalsh(C) > 0)


def sample(n, rng):
    L = (rng.random(n) >= P[0]).astype(int)          # P(L=1)=0.65
    X = np.empty((n, 4))
    X[L == 0] = rng.multivariate_normal(m0, C0, (L == 0).sum())
    X[L == 1] = rng.multivariate_normal(m1, C1, (L == 1).sum())
    return X, L


X, L = sample(N, rng)
np.savez("out/q1_data.npz", X=X, L=L)                # same data set used for every part
N0, N1 = (L == 0).sum(), (L == 1).sum()
res = {"N0": int(N0), "N1": int(N1)}

gamma_theory = P[0] / P[1]                           # 0-1 loss: (l10-l00)P0 / ((l01-l11)P1)
res["gamma_theory"] = gamma_theory


def analyse(score, name, theory_thr=None, priors=P):
    """score: decision statistic, D=1 iff score > thr. Returns dict + ROC arrays."""
    thr, fpr, tpr = roc_sweep(score, L)
    perr = fpr * priors[0] + (1 - tpr) * priors[1]
    k = int(np.argmin(perr))
    # any threshold in [thr_k, thr_{k+1}) gives the same decisions; report the midpoint
    t_lo = thr[k]
    t_hi = thr[k + 1] if k + 1 < len(thr) else np.inf
    t_mid = 0.5 * (t_lo + t_hi) if np.isfinite(t_lo) and np.isfinite(t_hi) else t_lo
    out = dict(name=name, k=k, thr_mid=float(t_mid), thr_lo=float(t_lo), thr_hi=float(t_hi),
               perr_min=float(perr[k]), tpr=float(tpr[k]), fpr=float(fpr[k]),
               auc=float(auc(fpr, tpr)), n_thr=len(thr))
    if theory_thr is not None:
        D = score > theory_thr
        out["theory_tpr"] = float(D[L == 1].mean())
        out["theory_fpr"] = float(D[L == 0].mean())
        out["theory_perr"] = float(out["theory_fpr"] * P[0] + (1 - out["theory_tpr"]) * P[1])
    return out, (fpr, tpr, perr, thr)


# ---------- Part A: true pdf ----------
llr_A = gauss_logpdf(X, m1, C1) - gauss_logpdf(X, m0, C0)          # ln p(x|1) - ln p(x|0)
rA, curA = analyse(llr_A, "True pdf (ERM)", np.log(gamma_theory))
rA["gamma_emp"] = float(np.exp(rA["thr_mid"]))
rA["gamma_emp_lo"] = float(np.exp(rA["thr_lo"]))
rA["gamma_emp_hi"] = float(np.exp(rA["thr_hi"]))

# ---------- Part B: naive Bayes (diagonal covariances with true variances) ----------
C0d, C1d = np.diag(np.diag(C0)), np.diag(np.diag(C1))
llr_B = gauss_logpdf(X, m1, C1d) - gauss_logpdf(X, m0, C0d)
rB, curB = analyse(llr_B, "Naive Bayes", np.log(gamma_theory))
rB["gamma_emp"] = float(np.exp(rB["thr_mid"]))
rB["gamma_emp_lo"] = float(np.exp(rB["thr_lo"]))
rB["gamma_emp_hi"] = float(np.exp(rB["thr_hi"]))
```

Source appendix | q1.py (continued)

```
 78
 79 # ---------- Part C: Fisher LDA from sample estimates ----------
 80 mu0_hat, mu1_hat = X[L == 0].mean(0), X[L == 1].mean(0)
 81 S0_hat = np.cov(X[L == 0].T, bias=True)          # sample-average (1/N_j) estimators
 82 S1_hat = np.cov(X[L == 1].T, bias=True)
 83 Sb = np.outer(mu1_hat - mu0_hat, mu1_hat - mu0_hat)
 84 Sw = S0_hat + S1_hat                             # equal weights, priors ignored (as in class)
 85 evals, evecs = eigh(Sb, Sw)                      # generalised eigenproblem Sb w = lambda Sw w
 86 w = evecs[:, np.argmax(evals)]
 87 w = w / np.linalg.norm(w)
 88 if w @ mu1_hat < w @ mu0_hat:                    # orient so that class 1 projects higher
 89     w = -w
 90 y = X @ w
 91 rC, curC = analyse(y, "Fisher LDA", priors=np.array([N0, N1]) / N)  # sample-count estimate
 92 rC["count_check"] = float(np.mean((y > rC["thr_mid"]) != L))
 93 w_closed = np.linalg.solve(Sw, mu1_hat - mu0_hat)
 94 w_closed /= np.linalg.norm(w_closed)
 95 res["lda"] = dict(w=w.tolist(), w_closed=w_closed.tolist(), eig=float(evals.max()),
 96                   mu0_hat=mu0_hat.tolist(), mu1_hat=mu1_hat.tolist(),
 97                   S0_hat=S0_hat.tolist(), S1_hat=S1_hat.tolist())
 98
 99 # ---------- reference: theoretical min P(error) via a large independent Monte Carlo ----------
100 Xbig, Lbig = sample(2_000_000, np.random.default_rng(1))
101 llr_big = gauss_logpdf(Xbig, m1, C1) - gauss_logpdf(Xbig, m0, C0)
102 res["perr_bayes_mc"] = float(np.mean((llr_big > np.log(gamma_theory)) != Lbig))
103 llr_bigB = gauss_logpdf(Xbig, m1, C1d) - gauss_logpdf(Xbig, m0, C0d)
104 res["perr_nb_mc_theorythr"] = float(np.mean((llr_bigB > np.log(gamma_theory)) != Lbig))
105
106 res["A"], res["B"], res["C"] = rA, rB, rC
107 for key, curve in [("A", curA), ("B", curB), ("C", curC)]:
108     fpr, tpr, perr, thresholds = curve
109     np.savez(f"out/q1_roc_{key}.npz", fpr=fpr, tpr=tpr, fnr=1-tpr, perr=perr, thresholds=thresholds)
110 res["perr_bayes_mc_se"] = float(np.sqrt(res["perr_bayes_mc"]*(1-res["perr_bayes_mc"])/len(Lbig)))
111
112 json.dump(res, open("out/q1_results.json", "w"), indent=2)
113
114 # ---------- figures ----------
115 def roc_plot(ax, cur, r, title, color, theory=True):
116     fpr, tpr, _, _ = cur
117     ax.plot(fpr, tpr, color=color, lw=1.6, label=f"ROC (AUC = {r['auc']:.4f})")
118     ax.plot(r["fpr"], r["tpr"], "o", ms=10, mfc="none", mec="red", mew=2.2,
119             label=f"Empirical min-P(error): P(e) = {r['perr_min']:.4f}")
120     if theory and "theory_tpr" in r:
121         ax.plot(r["theory_fpr"], r["theory_tpr"], "x", ms=10, color="black", mew=2.2,
122                 label=f"Theoretical $\\gamma^*=0.538$: P(e) = {r['theory_perr']:.4f}")
123     ax.plot([0, 1], [0, 1], ":", color="gray", lw=0.8)
124     ax.set_xlabel(r"False positive rate $P(D=1\mid L=0)$")
125     ax.set_ylabel(r"True positive rate $P(D=1\mid L=1)$")
126     ax.set_title(title)
127     ax.legend(loc="lower right", fontsize=8.5)
128     ax.grid(alpha=0.3)
129     ax.set_xlim(-0.01, 1.01); ax.set_ylim(-0.01, 1.01)
130
131
132 for cur, r, t, c, fn in [(curA, rA, "Part A: ERM with true pdf", "C0", "q1a_roc"),
133                          (curB, rB, "Part B: Naive Bayes (diagonal covariances)", "C2", "q1b_roc"),
134                          (curC, rC, r"Part C: Fisher LDA, $w^T x$ vs. $\tau$", "C1", "q1c_roc")]:
135     fig, ax = plt.subplots(figsize=(6.2, 5.2))
136     roc_plot(ax, cur, r, t, c)
137     # zoomed inset near the operating point
138     ins = ax.inset_axes([0.45, 0.32, 0.5, 0.36])
139     ins.plot(cur[0], cur[1], color=c, lw=1.4)
140     ins.plot(r["fpr"], r["tpr"], "o", ms=8, mfc="none", mec="red", mew=2)
141     if "theory_tpr" in r:
142         ins.plot(r["theory_fpr"], r["theory_tpr"], "x", ms=8, color="black", mew=2)
143     ins.set_xlim(0, 0.2); ins.set_ylim(0.8, 1.0); ins.grid(alpha=0.3); ins.tick_params(labelsize=7)
144     fig.tight_layout(); fig.savefig(f"figs/{fn}.png", dpi=170); plt.close(fig)
145
146 # comparison plot
147 fig, ax = plt.subplots(figsize=(6.2, 5.2))
148 for cur, r, c in [(curA, rA, "C0"), (curB, rB, "C2"), (curC, rC, "C1")]:
149     ax.plot(cur[0], cur[1], color=c, lw=1.5, label=f"{r['name']} (AUC {r['auc']:.4f}, min P(e)
    {r['perr_min']:.4f})")
150     ax.plot(r["fpr"], r["tpr"], "o", ms=8, mfc="none", mec=c, mew=2)
151 ax.set_xlim(0, 0.3); ax.set_ylim(0.7, 1.0); ax.grid(alpha=0.3)
152 ax.set_xlabel(r"$P(D=1\mid L=0)$"); ax.set_ylabel(r"$P(D=1\mid L=1)$")
153 ax.set_title("ROC comparison (zoomed); circles = min-P(error) points")
```

Source appendix | q1.py (continued)

```
ax.legend(fontsize=8, loc="lower right")
fig.tight_layout(); fig.savefig("figs/q1_compare.png", dpi=170); plt.close(fig)

# P(error) vs threshold
fig, axs = plt.subplots(1, 3, figsize=(13, 3.6))
for ax, cur, r, xl in [(axs[0], curA, rA, r"$\ln\gamma$"), (axs[1], curB, rB, r"$\ln\gamma$"),
                       (axs[2], curC, rC, r"$\tau$")]:
    _, _, perr, thr = cur
    ok = np.isfinite(thr)
    ax.plot(thr[ok], perr[ok], lw=1.2)
    ax.axvline(r["thr_mid"], color="red", ls="--", lw=1, label=f"empirical min at {r['thr_mid']:.3f}")
    if r is not rC:
        ax.axvline(np.log(gamma_theory), color="k", ls=":", lw=1.2, label=fr"$\ln\gamma^*$ =
{np.log(gamma_theory):.3f}")
    lo, hi = np.percentile(thr[ok], [2, 98])
    ax.set_xlim(lo, hi); ax.set_xlabel(xl); ax.set_ylabel("P(error)"); ax.set_title(r["name"])
    ax.legend(fontsize=8); ax.grid(alpha=0.3)
fig.tight_layout(); fig.savefig("figs/q1_perr_vs_thr.png", dpi=160); plt.close(fig)

print(json.dumps({k: v for k, v in res.items() if k != "lda"}, indent=1))
print("w", w, "w_closed", w_closed)
```

Source appendix | q2.py

```python
"""Question 2: 3-class, 3-D Gaussian-mixture problem; MAP (0-1 loss) and ERM with Lambda_10, Lambda_100."""
import json
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
from common import gauss_logpdf

rng = np.random.default_rng(2026)
N = 10000
priors = np.array([0.3, 0.3, 0.4])

# Four component means on a regular tetrahedron with edge length a = 2.5
a = 2.5
means = np.array([[0, 0, 0],
                  [a, 0, 0],
                  [a / 2, a * np.sqrt(3) / 2, 0],
                  [a / 2, a * np.sqrt(3) / 6, a * np.sqrt(2 / 3)]])
covs = np.array([[[1.0, 0.3, -0.2], [0.3, 0.8, 0.1], [-0.2, 0.1, 1.2]],
                 [[0.9, -0.2, 0.1], [-0.2, 1.1, 0.3], [0.1, 0.3, 0.8]],
                 [[1.2, 0.2, 0.0], [0.2, 0.9, -0.3], [0.0, -0.3, 1.0]],
                 [[0.8, 0.0, 0.2], [0.0, 1.0, 0.0], [0.2, 0.0, 1.1]]])
for C in covs:
    assert np.all(np.linalg.eigvalsh(C) > 0)
# class 1 <- component 0, class 2 <- component 1, class 3 <- 0.5*comp 2 + 0.5*comp 3
avg_std = np.mean([np.sqrt(np.trace(C) / 3) for C in covs])
dists = [np.linalg.norm(means[i] - means[j]) for i in range(4) for j in range(i + 1, 4)]

# ---- 1. generate data ----
L = rng.choice([1, 2, 3], size=N, p=priors)
comp = np.where(L == 1, 0, np.where(L == 2, 1, 2 + (rng.random(N) < 0.5)))
X = np.empty((N, 3))
for c in range(4):
    idx = comp == c
    X[idx] = rng.multivariate_normal(means[c], covs[c], idx.sum())
np.savez("out/q2_data.npz", X=X, L=L, comp=comp)


def class_loglik(X):
    """N x 3 matrix of ln p(x|L=j)."""
    l1 = gauss_logpdf(X, means[0], covs[0])
    l2 = gauss_logpdf(X, means[1], covs[1])
    l3 = np.logaddexp(np.log(0.5) + gauss_logpdf(X, means[2], covs[2]),
                      np.log(0.5) + gauss_logpdf(X, means[3], covs[3]))
    return np.c_[l1, l2, l3]


ll = class_loglik(X)
lj = ll + np.log(priors)                                    # ln p(x|j)P(j)
post = np.exp(lj - np.logaddexp.reduce(lj, axis=1, keepdims=True))   # P(L=j|x)

Lambda = {"0-1": 1 - np.eye(3),
          "10": np.array([[0, 1, 10], [1, 0, 10], [1, 1, 0]], float),
          "100": np.array([[0, 1, 100], [1, 0, 100], [1, 1, 0]], float)}

res = {"means": means.tolist(), "covs": covs.tolist(), "avg_std": avg_std,
       "pairwise_dist": dists, "class_counts": [int((L == j).sum()) for j in (1, 2, 3)]}

for key, Lam in Lambda.items():
    R = post @ Lam.T                       # R[n,i] = sum_j Lam[i,j] P(j|x_n)  (conditional risk)
    D = np.argmin(R, axis=1) + 1
    conf = np.zeros((3, 3))
    counts = np.zeros((3, 3), int)
    for i in (1, 2, 3):
        for j in (1, 2, 3):
            counts[i - 1, j - 1] = np.sum((D == i) & (L == j))
            conf[i - 1, j - 1] = counts[i - 1, j - 1] / np.sum(L == j)
    risk = float(np.mean(Lam[D - 1, L - 1]))            # sample average of incurred loss
    perr = float(np.mean(D != L))
    res[key] = dict(conf=conf.tolist(), counts=counts.tolist(), risk=risk, perr=perr,
                    dec_counts=[int((D == i).sum()) for i in (1, 2, 3)],
                    exp_risk_cond=float(np.mean(R.min(axis=1))))
    np.save(f"out/q2_D_{key}.npy", D)

    # ---- 3-D scatter plot ----
    fig = plt.figure(figsize=(12, 6.2))
```

Source appendix | q2.py (continued)

```
    markers = {1: ".", 2: "^", 3: "s"}
    sizes = {1: 30, 2: 10, 3: 8}
    ok = D == L
    for p, (elev, azim) in enumerate([(22, -60), (12, 30)]):
        ax = fig.add_subplot(1, 2, p + 1, projection="3d")
        for j in (1, 2, 3):
            for good, col in ((True, "green"), (False, "red")):
                idx = (L == j) & (ok == good)
                ax.scatter(*X[idx].T, marker=markers[j], s=sizes[j],
                           c=col, alpha=0.35 if good else 0.8, linewidths=0.3)
        ax.view_init(elev, azim)
        ax.set_box_aspect(None, zoom=1.0)
        ax.set_xlabel("$x_1$"); ax.set_ylabel("$x_2$"); ax.set_zlabel("$x_3$")
    handles = [Line2D([], [], marker=markers[j], ls="", color="gray", ms=7, label=f"L = {j}") for j in (1,
2, 3)]
    handles += [Line2D([], [], marker="o", ls="", color="green", label="correct"),
                Line2D([], [], marker="o", ls="", color="red", label="incorrect")]
    name = "MAP (0-1 loss)" if key == "0-1" else fr"ERM with $\Lambda_{{{key}}}$"
    fig.suptitle(f"{name}: P(error) = {perr:.4f}, estimated min. expected risk = {risk:.4f}")
    fig.legend(handles=handles, loc="lower center", ncol=5, fontsize=9)
    fig.subplots_adjust(left=0.02, right=0.98, top=0.95, bottom=0.07, wspace=0.08)
    fig.savefig(f"figs/q2_scatter_{key.replace('-', '')}.png", dpi=150); plt.close(fig)

# ---- confusion-matrix heat maps side by side ----
fig, axs = plt.subplots(1, 3, figsize=(12, 3.8))
for ax, key in zip(axs, Lambda):
    M = np.array(res[key]["conf"])
    ax.imshow(M, cmap="Blues", vmin=0, vmax=1)
    for i in range(3):
        for j in range(3):
            ax.text(j, i, f"{M[i, j]:.3f}", ha="center", va="center",
                    color="white" if M[i, j] > 0.6 else "black", fontsize=10)
    ax.set_xticks(range(3), ["L=1", "L=2", "L=3"]); ax.set_yticks(range(3), ["D=1", "D=2", "D=3"])
    t = "0-1 loss (MAP)" if key == "0-1" else fr"$\Lambda_{{{key}}}$"
    ax.set_title(f"{t}\nrisk {res[key]['risk']:.4f}, P(e) {res[key]['perr']:.4f}", fontsize=10)
fig.suptitle(r"Confusion matrices $P(D=i\mid L=j)$ (columns sum to 1)")
fig.tight_layout(); fig.savefig("figs/q2_confusions.png", dpi=150); plt.close(fig)

# ---- risk/error trade-off as the class-3 loss weight grows ----
ws = np.r_[1, 2, 3, 5, 7, 10, 15, 20, 30, 50, 70, 100, 200, 500]
rk, pe, p3 = [], [], []
for wgt in ws:
    Lam = np.array([[0, 1, wgt], [1, 0, wgt], [1, 1, 0]], float)
    D = np.argmin(post @ Lam.T, axis=1) + 1
    rk.append(np.mean(Lam[D - 1, L - 1])); pe.append(np.mean(D != L)); p3.append(np.mean(D == 3))
res["sweep"] = dict(w=ws.tolist(), risk=rk, perr=pe, frac_D3=p3)
fig, ax = plt.subplots(figsize=(6.4, 4))
ax.semilogx(ws, pe, "o-", label="P(error)")
ax.semilogx(ws, p3, "s-", label="fraction decided D=3")
ax.semilogx(ws, rk, "^-", label="estimated expected risk")
ax.set_xlabel("loss for misclassifying a class-3 sample (w)"); ax.grid(alpha=0.3, which="both")
ax.legend(); ax.set_title("Effect of the class-3 loss weight")
fig.tight_layout(); fig.savefig("figs/q2_sweep.png", dpi=150); plt.close(fig)

json.dump(res, open("out/q2_results.json", "w"), indent=2)
for key in Lambda:
    print(key, res[key]["risk"], res[key]["perr"], res[key]["dec_counts"])
    print(np.round(np.array(res[key]["conf"]), 4))
print("avg std", avg_std, "dists", np.round(dists, 3), res["class_counts"])
```

Source appendix | q3.py

```
"""Question 3: Gaussian class-conditional (QDA-type) minimum-P(error) classifiers on real data."""
import json
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy.stats import chi2
from common import gauss_logpdf

from pathlib import Path
from scipy.linalg import solve_triangular
DATA = Path(__file__).resolve().parent / "data"


# ---------------------------------------------------------------- data
def load_wine():
    df = pd.read_csv(f"{DATA}/wine_white.csv", sep=";")
    X = df.drop(columns="quality").to_numpy(float)
    y = df["quality"].to_numpy(int)
    return X, y, list(df.columns[:-1])


def load_har():
    Xtr = np.loadtxt(f"{DATA}/har/X_train.txt"); Xte = np.loadtxt(f"{DATA}/har/X_test.txt")
    ytr = np.loadtxt(f"{DATA}/har/y_train.txt", int); yte = np.loadtxt(f"{DATA}/har/y_test.txt", int)
    names = [l.split()[1] for l in open(f"{DATA}/har/activity_labels.txt")]
    return np.vstack([Xtr, Xte]), np.r_[ytr, yte], names, len(ytr)


# ---------------------------------------------------------------- model
def fit(X, y, alpha, diagnostics=False):
    """Per-class priors (counts), sample-average means and covariances, plus
    lambda_j = alpha * trace(C_j) / rank(C_j) added to the diagonal."""
    model = {}
    for c in np.unique(y):
        Xc = X[y == c]
        m = Xc.mean(0)
        C = (Xc - m).T @ (Xc - m) / len(Xc)            # 1/N_c sample-average estimator
        eig = np.linalg.eigvalsh(C)
        tol = max(C.shape) * np.finfo(float).eps * max(abs(eig))
        r = int(np.sum(eig > tol))
        if r == 0:
            raise ValueError("Zero-rank class covariance requires a separate variance floor")
        lam = alpha * np.trace(C) / r
        model[c] = dict(prior=len(Xc) / len(X), m=m, C=C + lam * np.eye(X.shape[1]), lam=lam,
                        rank=int(r), n=len(Xc), cond_raw=float(np.linalg.cond(C)) if diagnostics else None,
                        cond_reg=float(np.linalg.cond(C + lam * np.eye(X.shape[1]))) if diagnostics else
None)
    return model


def predict(model, X):
    cls = np.array(sorted(model))
    S = np.column_stack([gauss_logpdf(X, model[c]["m"], model[c]["C"]) + np.log(model[c]["prior"]) for c in
 cls])
    return cls[np.argmax(S, 1)]


def confusion(D, y, cls):
    """M[i,j] = P(D=cls_i | L=cls_j), counts as well."""
    K = len(cls)
    cnt = np.zeros((K, K), int)
    for a, ci in enumerate(cls):
        for b, cj in enumerate(cls):
            cnt[a, b] = np.sum((D == ci) & (y == cj))
    return cnt / cnt.sum(0, keepdims=True), cnt


def stratified_folds(y, k, rng):
    f = np.empty(len(y), int)
    for c in np.unique(y):
        idx = rng.permutation(np.where(y == c)[0])
        f[idx] = np.arange(len(idx)) % k
    return f


```

Source appendix | q3.py (continued)

```
def heat(ax, M, cnt, ticks, title):
    ax.imshow(M, cmap="Blues", vmin=0, vmax=1)
    for i in range(M.shape[0]):
        for j in range(M.shape[1]):
            if cnt[:, j].sum() == 0:
                continue
            ax.text(j, i, f"{M[i, j]:.2f}\n({cnt[i, j]})", ha="center", va="center", fontsize=7,
                    color="white" if M[i, j] > 0.6 else "black")
    ax.set_xticks(range(len(ticks)), [f"L={t}" for t in ticks], fontsize=8, rotation=0)
    ax.set_yticks(range(len(ticks)), [f"D={t}" for t in ticks], fontsize=8)
    ax.set_title(title, fontsize=10)


def pca(X, k):
    Xc = X - X.mean(0)
    U, s, Vt = np.linalg.svd(Xc, full_matrices=False)
    return Xc @ Vt[:k].T, s ** 2 / np.sum(s ** 2)


res = {}
rng = np.random.default_rng(0)
alphas = [1e-6, 1e-5, 1e-4, 1e-3, 1e-2, 3e-2, 1e-1, 3e-1]

# ================================================================ WINE
Xw_raw, yw, wnames = load_wine()
mu_w, sd_w = Xw_raw.mean(0), Xw_raw.std(0)
Xw = (Xw_raw - mu_w) / sd_w                    # z-score (see report for why)
cls_w = np.unique(yw)
res["wine_counts"] = {int(c): int((yw == c).sum()) for c in cls_w}

folds = stratified_folds(yw, 5, rng)
sweep_w = []
for a in alphas:
    m = fit(Xw, yw, a)
    tr_err = float(np.mean(predict(m, Xw) != yw))
    cv_err = []
    for f in range(5):
        # Fit preprocessing exclusively on this fold's training portion.
        train, valid = Xw_raw[folds != f], Xw_raw[folds == f]
        center, scale = train.mean(0), train.std(0)
        scale = np.where(scale > 0, scale, 1.0)
        mf = fit((train-center)/scale, yw[folds != f], a)
        cv_err.append(np.mean(predict(mf, (valid-center)/scale) != yw[folds == f]))
    sweep_w.append(dict(alpha=a, train=tr_err, cv=float(np.mean(cv_err))))
res["wine_sweep"] = sweep_w
alpha_w = min(sweep_w, key=lambda r: r["cv"])["alpha"]
mw = fit(Xw, yw, alpha_w, diagnostics=True)
Dw = predict(mw, Xw)
Mw, Cw = confusion(Dw, yw, cls_w)
res["wine"] = dict(alpha=alpha_w, perr=float(np.mean(Dw != yw)), conf=Mw.tolist(), counts=Cw.tolist(),
                   classes=cls_w.tolist(),
                   per_class={int(c): dict(n=mw[c]["n"], prior=mw[c]["prior"], rank=mw[c]["rank"],
                                           lam=mw[c]["lam"], cond_raw=mw[c]["cond_raw"],
                                           cond_reg=mw[c]["cond_reg"]) for c in cls_w},
                   majority_err=float(1 - max(np.mean(yw == c) for c in cls_w)))
# same classifier on raw (unstandardised) features, for comparison
mw_raw = fit(Xw_raw, yw, alpha_w)
res["wine"]["perr_raw_features"] = float(np.mean(predict(mw_raw, Xw_raw) != yw))
# within-one-grade accuracy
res["wine"]["within1"] = float(np.mean(np.abs(Dw - yw) <= 1))

# ================================================================ HAR
Xh, yh, hnames, ntr = load_har()
mu_h, sd_h = Xh.mean(0), Xh.std(0)
Xhs = (Xh - mu_h) / sd_h
cls_h = np.unique(yh)
res["har_counts"] = {int(c): int((yh == c).sum()) for c in cls_h}
sweep_h = []
for a in alphas:
    m = fit(Xhs, yh, a)
    tr_err = float(np.mean(predict(m, Xhs) != yh))
    # held-out check on the dataset's own subject-disjoint train/test split
    # This official test split is used as validation for alpha, not as an unbiased final test.
    center, scale = Xh[:ntr].mean(0), Xh[:ntr].std(0)
    scale = np.where(scale > 0, scale, 1.0)
    mt = fit((Xh[:ntr]-center)/scale, yh[:ntr], a)
    te_err = float(np.mean(predict(mt, (Xh[ntr:]-center)/scale) != yh[ntr:]))
```

Source appendix | q3.py (continued)

```
    sweep_h.append(dict(alpha=a, train=tr_err, test=te_err))
    print("HAR", a, tr_err, te_err, flush=True)
res["har_sweep"] = sweep_h
alpha_h = min(sweep_h, key=lambda r: r["test"])["alpha"]
mh = fit(Xhs, yh, alpha_h, diagnostics=True)
Dh = predict(mh, Xhs)
Mh, Ch = confusion(Dh, yh, cls_h)
res["har"] = dict(alpha=alpha_h, perr=float(np.mean(Dh != yh)), conf=Mh.tolist(), counts=Ch.tolist(),
                  names=hnames,
                  per_class={int(c): dict(n=mh[c]["n"], prior=mh[c]["prior"], rank=mh[c]["rank"],
                                          lam=mh[c]["lam"], cond_raw=mh[c]["cond_raw"],
                                          cond_reg=mh[c]["cond_reg"]) for c in cls_h})
json.dump(res, open("out/q3_results.json", "w"), indent=2, default=float)

# ============================================================== FIGURES
# confusion matrices
fig, ax = plt.subplots(figsize=(6.4, 5.4))
heat(ax, Mw, Cw, cls_w, fr"Wine (white): $P(D=i\mid L=j)$, P(error) = {res['wine']['perr']:.4f}")
fig.tight_layout(); fig.savefig("figs/q3_wine_conf.png", dpi=160); plt.close(fig)
fig, ax = plt.subplots(figsize=(6.4, 5.4))
heat(ax, Mh, Ch, cls_h, fr"HAR: $P(D=i\mid L=j)$, P(error) = {res['har']['perr']:.4f}")
fig.tight_layout(); fig.savefig("figs/q3_har_conf.png", dpi=160); plt.close(fig)

# regularisation sweep
fig, axs = plt.subplots(1, 2, figsize=(11, 3.8))
axs[0].semilogx(alphas, [r["train"] for r in sweep_w], "o-", label="all-sample (training) error")
axs[0].semilogx(alphas, [r["cv"] for r in sweep_w], "s-", label="5-fold CV error")
axs[0].axvline(alpha_w, color="k", ls=":"); axs[0].set_title("Wine"); axs[0].set_xlabel(r"$\alpha$")
axs[1].semilogx(alphas, [r["train"] for r in sweep_h], "o-", label="all-sample (training) error")
axs[1].semilogx(alphas, [r["test"] for r in sweep_h], "s-", label="validation error (official split; used
    to select alpha)")
axs[1].axvline(alpha_h, color="k", ls=":"); axs[1].set_title("HAR"); axs[1].set_xlabel(r"$\alpha$")
for a in axs:
    a.set_ylabel("P(error)"); a.grid(alpha=0.3, which="both"); a.legend(fontsize=8)
fig.tight_layout(); fig.savefig("figs/q3_alpha_sweep.png", dpi=160); plt.close(fig)

# PCA projections
cmap = plt.get_cmap("tab10")
for nm, X, y, cls, lab in [("wine", Xw, yw, cls_w, lambda c: f"q={c}"),
                           ("har", Xhs, yh, cls_h, lambda c: hnames[c - 1])]:
    Z, evr = pca(X, 3)
    fig = plt.figure(figsize=(13, 4.6))
    ax1 = fig.add_subplot(1, 3, 1); ax2 = fig.add_subplot(1, 3, 2); ax3 = fig.add_subplot(1, 3, 3,
    projection="3d")
    for k, c in enumerate(cls):
        i = y == c
        kw = dict(s=4, alpha=0.45, color=cmap(k), label=lab(c))
        ax1.scatter(Z[i, 0], Z[i, 1], **kw); ax2.scatter(Z[i, 0], Z[i, 2], **kw)
        ax3.scatter(Z[i, 0], Z[i, 1], Z[i, 2], **kw)
    ax1.set_xlabel(f"PC1 ({evr[0]:.1%})"); ax1.set_ylabel(f"PC2 ({evr[1]:.1%})")
    ax2.set_xlabel("PC1"); ax2.set_ylabel(f"PC3 ({evr[2]:.1%})")
    ax3.set_xlabel("PC1"); ax3.set_ylabel("PC2"); ax3.set_zlabel("PC3")
    ax1.legend(markerscale=4, fontsize=7, loc="best")
    res[nm.lower() + "_pca_explained"] = evr[:3].tolist()
    fig.suptitle(f"{'White wine' if nm == 'wine' else 'HAR'}: first three principal components
    (standardised features)")
    fig.tight_layout(); fig.savefig(f"figs/q3_{nm}_pca.png", dpi=150); plt.close(fig)

# wine: feature-subset scatter + marginal histograms
fig, axs = plt.subplots(1, 3, figsize=(13, 4))
pairs = [("alcohol", "density"), ("alcohol", "volatile acidity"), ("residual sugar", "chlorides")]
for ax, (fx, fy) in zip(axs, pairs):
    ix, iy = wnames.index(fx), wnames.index(fy)
    for k, c in enumerate(cls_w):
        i = yw == c
        ax.scatter(Xw_raw[i, ix], Xw_raw[i, iy], s=4, alpha=0.5, color=cmap(k), label=f"q={c}")
    ax.set_xlabel(fx); ax.set_ylabel(fy)
axs[0].legend(markerscale=4, fontsize=7)
fig.suptitle("White wine: raw feature pairs")
fig.tight_layout(); fig.savefig("figs/q3_wine_pairs.png", dpi=150); plt.close(fig)

fig, axs = plt.subplots(2, 6, figsize=(15, 5))
for k, ax in enumerate(axs.ravel()):
    if k >= len(wnames):
        ax.axis("off"); continue
    ax.hist(Xw_raw[yw == 6, k], bins=50, density=True, color="C0", alpha=0.7)
    ax.set_title(wnames[k], fontsize=9); ax.tick_params(labelsize=7)
```

Source appendix | q3.py (continued)

```
fig.suptitle("White wine, quality-6 class: marginal histograms (a Gaussian model implies symmetric, bell-shaped marginals)")
fig.tight_layout(); fig.savefig("figs/q3_wine_hist.png", dpi=140); plt.close(fig)

# HAR: a few raw features
fig, axs = plt.subplots(1, 2, figsize=(11, 4.2))
for ax, (fx, fy) in zip(axs, [(0, 1), (40, 52)]):   # 0-based column indices into the 561 features
    for k, c in enumerate(cls_h):
        i = yh == c
        ax.scatter(Xh[i, fx], Xh[i, fy], s=3, alpha=0.4, color=cmap(k), label=hnames[c - 1])
    ax.set_xlabel(f"feature {fx + 1}"); ax.set_ylabel(f"feature {fy + 1}")
axs[0].legend(markerscale=4, fontsize=7)
fig.suptitle("HAR: two raw feature pairs")
fig.tight_layout(); fig.savefig("figs/q3_har_pairs.png", dpi=150); plt.close(fig)

# Gaussianity check: squared Mahalanobis distances vs chi^2_d quantiles (Q-Q)
fig, axs = plt.subplots(1, 2, figsize=(11, 4.3))
for ax, (nm, X, y, mdl, cs) in zip(axs, [("Wine", Xw, yw, mw, [5, 6, 7]), ("HAR", Xhs, yh, mh, [1, 4, 6])]):
    d = X.shape[1]
    for c in cs:
        Xc = X[y == c]
        m = Xc.mean(0); C = mdl[c]["C"]
        Lc = np.linalg.cholesky(C)
        r2 = np.sort(np.sum(solve_triangular(Lc, (Xc - m).T, lower=True, check_finite=False) ** 2, 0))
        q = chi2.ppf((np.arange(1, len(r2) + 1) - 0.5) / len(r2), d)
        lab = f"q={c}" if nm == "Wine" else hnames[c - 1]
        ax.plot(q, r2, ".", ms=2, label=lab)
    lim = ax.get_xlim()
    ax.plot(lim, lim, "k--", lw=1, label="y = x (Gaussian)")
    ax.set_xlabel(fr"$\chi^2_{{{d}}}$ quantiles"); ax.set_ylabel("sorted squared Mahalanobis distance")
    ax.set_title(f"{nm}: descriptive Q-Q diagnostic (fitted, regularized)"); ax.legend(markerscale=5, fontsize=8)
    if nm == "Wine":
        ax.set_ylim(0, 150)
    ax.grid(alpha=0.3)
fig.tight_layout(); fig.savefig("figs/q3_qq.png", dpi=150); plt.close(fig)

print(json.dumps({k: res[k] for k in ("wine_sweep", "har_sweep")}, indent=1))
print("wine", alpha_w, res["wine"]["perr"], res["wine"]["perr_raw_features"], res["wine"]["within1"], res["wine"]["majority_err"])
print(np.round(Mw, 3)); print(Cw)
print("har", alpha_h, res["har"]["perr"]); print(np.round(Mh, 4)); print(Ch)
for c in cls_w: print(c, res["wine"]["per_class"][int(c)])
for c in cls_h: print(c, res["har"]["per_class"][int(c)])

json.dump(res, open("out/q3_results.json", "w"), indent=2, default=float)
```

Source appendix | download_data.py

```
from pathlib import Path
import urllib.request,zipfile,io,hashlib,json
root=Path(__file__).resolve().parent
for d in ['data','out','figs','logs']: (root/d).mkdir(exist_ok=True)
manifest=[]
for name,url in [('wine','https://archive.ics.uci.edu/static/public/186/wine+quality.zip'),('har','https://
archive.ics.uci.edu/static/public/240/human+activity+recognition+using+smartphones.zip')]:
 target=root/'data'/f'{name}_source.zip'
 if not target.exists():
  print('Downloading',url,flush=True)
  with urllib.request.urlopen(url,timeout=120) as response: target.write_bytes(response.read())
 raw=target.read_bytes(); z=zipfile.ZipFile(io.BytesIO(raw))
 if name=='wine':
  entry=next(n for n in z.namelist() if n.endswith('winequality-white.csv'))
  (root/'data'/'wine_white.csv').write_bytes(z.read(entry))
 else:
  nested=[n for n in z.namelist() if n.endswith('.zip')]
  if nested: z=zipfile.ZipFile(io.BytesIO(z.read(nested[0])))
  dest=root/'data'/'har'; dest.mkdir(exist_ok=True)
  for base in ['X_train.txt','X_test.txt','y_train.txt','y_test.txt','activity_labels.txt']:
   entry=next(n for n in z.namelist() if n.endswith('/'+base) or n==base)
   (dest/base).write_bytes(z.read(entry))
 manifest.append(dict(dataset=name,url=url,sha256=hashlib.sha256(raw).hexdigest(),bytes=len(raw)))
 print(name,len(raw),'archive bytes; extracted',flush=True)
(root/'data'/'sources.json').write_text(json.dumps(manifest,indent=2))
```

Source appendix | run_all.py

```
"""Run all experiments from any working directory; outputs are saved beside this file."""
from pathlib import Path
import os
import subprocess
import sys
ROOT = Path(__file__).resolve().parent
for key in ['OPENBLAS_NUM_THREADS', 'OMP_NUM_THREADS', 'MKL_NUM_THREADS']:
    os.environ.setdefault(key, '2')
os.chdir(ROOT)
for name in ['data', 'out', 'figs', 'logs']:
    (ROOT / name).mkdir(exist_ok=True)
if not (ROOT / 'data/wine_white.csv').exists() or not (ROOT / 'data/har/X_train.txt').exists():
    subprocess.run([sys.executable, 'download_data.py'], check=True)
for name in ['q1.py', 'q2.py', 'q3.py', 'verify_results.py']:
    print('Running', name, flush=True)
    with open(ROOT / 'logs' / (Path(name).stem + '.log'), 'w', encoding='utf-8') as log:
        subprocess.run([sys.executable, name], stdout=log, stderr=subprocess.STDOUT, check=True)
print('Finished. See out/ for results and figs/ for figures. Run build_report.py separately to rebuild the
PDF.')
```

Source appendix | verify_results.py

```
"""Independent checks of numerical helpers and saved assignment results."""
import json, ast
from pathlib import Path
import numpy as np
from scipy.stats import multivariate_normal, rankdata
from common import gauss_logpdf, roc_sweep, auc
root=Path(__file__).resolve().parent
checks=[]
def check(name,condition):
 if not condition: raise AssertionError(name)
 checks.append(name)
rng=np.random.default_rng(4)
A=rng.normal(size=(4,4));C=A@A.T+np.eye(4);m=rng.normal(size=4);X=rng.normal(size=(30,4))
check('Gaussian log-density agrees with scipy.stats',np.allclose(gauss_logpdf(X,m,C),multivariate_normal.logpdf(X,m,C),atol=1e-11))
scores=np.array([0.,0.,1.,2.,2.,3.]);labels=np.array([0,1,0,1,0,1])
t,f,tpr=roc_sweep(scores,labels)
check('Tied-score ROC matches direct decisions at every threshold',all(np.isclose(f[i],np.mean((scores>v)[labels==0])) and np.isclose(tpr[i],np.mean((scores>v)[labels==1])) for i,v in enumerate(t)))
ranks=rankdata(scores);u=(ranks[labels==1].sum()-3*4/2)/9
check('ROC area agrees with independent rank statistic',np.isclose(auc(f,tpr),u))
r=json.loads((root/'out/q1_results.json').read_text())
for key in ['A','B','C']:
 c=np.load(root/f'out/q1_roc_{key}.npz');check(f'Q1 {key}: ROC endpoints and monotonicity',c['fpr'][0]==1 and c['tpr'][0]==1 and c['fpr'][-1]==0 and c['tpr'][-1]==0 and np.all(np.diff(c['fpr'])<=0) and np.all(np.diff(c['tpr'])<=0))
 check(f'Q1 {key}: AUC agrees with saved ROC',np.isclose(auc(c['fpr'],c['tpr']),r[key]['auc']))
check('LDA generalized eigenvector agrees with closed form',np.allclose(r['lda']['w'],r['lda']['w_closed']))
r2=json.loads((root/'out/q2_results.json').read_text())
for key,w in [('0-1',1),('10',10),('100',100)]:
 v=r2[key];M=np.array(v['conf']);cnt=np.array(v['counts']);Lam=np.array([[0,1,w],[1,0,w],[1,1,0]])
 check(f'Q2 {key}: columns normalized and 10000 counts',np.allclose(M.sum(0),1) and cnt.sum()==10000)
 check(f'Q2 {key}: risk agrees with confusion counts',np.isclose((cnt*Lam).sum()/10000,v['risk']))
r3=json.loads((root/'out/q3_results.json').read_text())
for key,n in [('wine',4898),('har',10299)]:
 v=r3[key];cnt=np.array(v['counts']);check(f'Q3 {key}: error and normalized confusion agree',cnt.sum()==n and np.isclose(1-np.trace(cnt)/n,v['perr']) and np.allclose(np.array(v['conf']).sum(0),1))
 check(f'Q3 {key}: positive regularization per class',all(c['lam']>0 for c in v['per_class'].values()))
# Inspect executable preprocessing, not just saved values.
s=(root/'q3.py').read_text()
check('Q3 CV scaling is fitted on fold training data','center, scale = train.mean(0), train.std(0)' in s)
check('Q3 official-split scaling is fitted on training data','center, scale = Xh[:ntr].mean(0), Xh[:ntr].std(0)' in s)
(root/'out/verification.json').write_text(json.dumps({'passed':len(checks),'checks':checks},indent=2))
print(f'{len(checks)} checks passed')
```